SET 3

PART A — Short questions and answers

Q1. Describe how a composite attribute such as Applicant_Address is represented in an ER diagram.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q2. Describe how a UNIQUE constraint on Email differs from a PRIMARY KEY.

Answer: UNIQUE prevents duplicate values in the specified column or column combination.

Q3. Summarize the conditions a relation must satisfy to be in First Normal Form (1NF).

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q4. Describe how a RIGHT OUTER JOIN between APPLICATION and PROGRAM lists programs that have no applications.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q5. Describe how AVG treats NULL values in Entrance_Score.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q6. Compare a procedure and a function in PL/SQL.

Answer: PL/SQL adds procedural programming to SQL. A procedure is a reusable named block that can take parameters.

Q7. Compare a row in a table with a document in a collection.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q8. Explain the role of the _id field in a document.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q9. Describe the output of the MongoDB query: db.applicants.find({City: "Chennai"}, {ApplicantName: 1, _id: 0})

Answer: The query returns all documents that satisfy the specified filter condition.

Q10. Interpret the Cypher query: MATCH (a:Applicant) RETURN a.name;

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

PART B — System solution

Scenario: A university maintains a University Admission System to store applicant details, programme details, and

1(a) ER design:

• **APPLICANT**(ApplicantID, ApplicantName, EntranceScore, Category, City); primary key: **ApplicantID**.

• **PROGRAM**(ProgramID, ProgramName, Department, Seats); primary key: **ProgramID**.

• **APPLICATION**(ApplicationID, ApplicantID, ProgramID, ApplicationDate, Status); primary key: **ApplicationID**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

1(b) SQL DDL/DML: Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE APPLICANT (
  ApplicantID NUMBER PRIMARY KEY,
  ApplicantName VARCHAR2(100),
  EntranceScore VARCHAR2(100),
  Category VARCHAR2(100),
  City VARCHAR2(100)
```

```
);
CREATE TABLE PROGRAM (
  ProgramID NUMBER PRIMARY KEY,
  ProgramName VARCHAR2(100),
  Department VARCHAR2(100),
  Seats NUMBER
);
CREATE TABLE APPLICATION (
  ApplicationID NUMBER PRIMARY KEY,
  ApplicantID NUMBER,
  ProgramID NUMBER,
  ApplicationDate DATE,
  Status VARCHAR2(100)
);
```

2(a) Join query: Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM APPLICATION x
JOIN APPLICANT a ON x.ApplicantID = a.ApplicantID
JOIN PROGRAM b ON x.ProgramID = b.ProgramID;
```

2(b) Aggregate query: Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

2(c) PL/SQL procedure: Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

3. MongoDB: Create the collections and insert documents using the fields listed in the set. Query with find() and the requested filter.

```
db.collection.insertMany([
  { _id: 1, name: 'Sample A', category: 'Example', price: 500 },
  { _id: 2, name: 'Sample B', category: 'Example', price: 300 }
]);
db.collection.find({ author: 'Specified Author' });
db.collection.find({ price: { $gt: 400 } });
```